

Build your movie *recommendation assistant.*

Choose a movie. Explain the choice.

Build a small product you can inspect and test.

🕒 15-20 min core reading · 30-45 min project practice · 12 short sections

Build something you can use, inspect, and test. Our first project recommends a movie from a small catalog, explains the choice, and handles the cases where a useful answer is not available.

WHAT YOU WILL BUILD

A complete path: preferences → eligible movies → ranked choice → an explanation with known and missing facts.

Two ways to run it: a working browser demo and a downloadable Python project. Both start with explicit rules; Python can optionally add an LLM-written draft using Chapter 10's adapter.

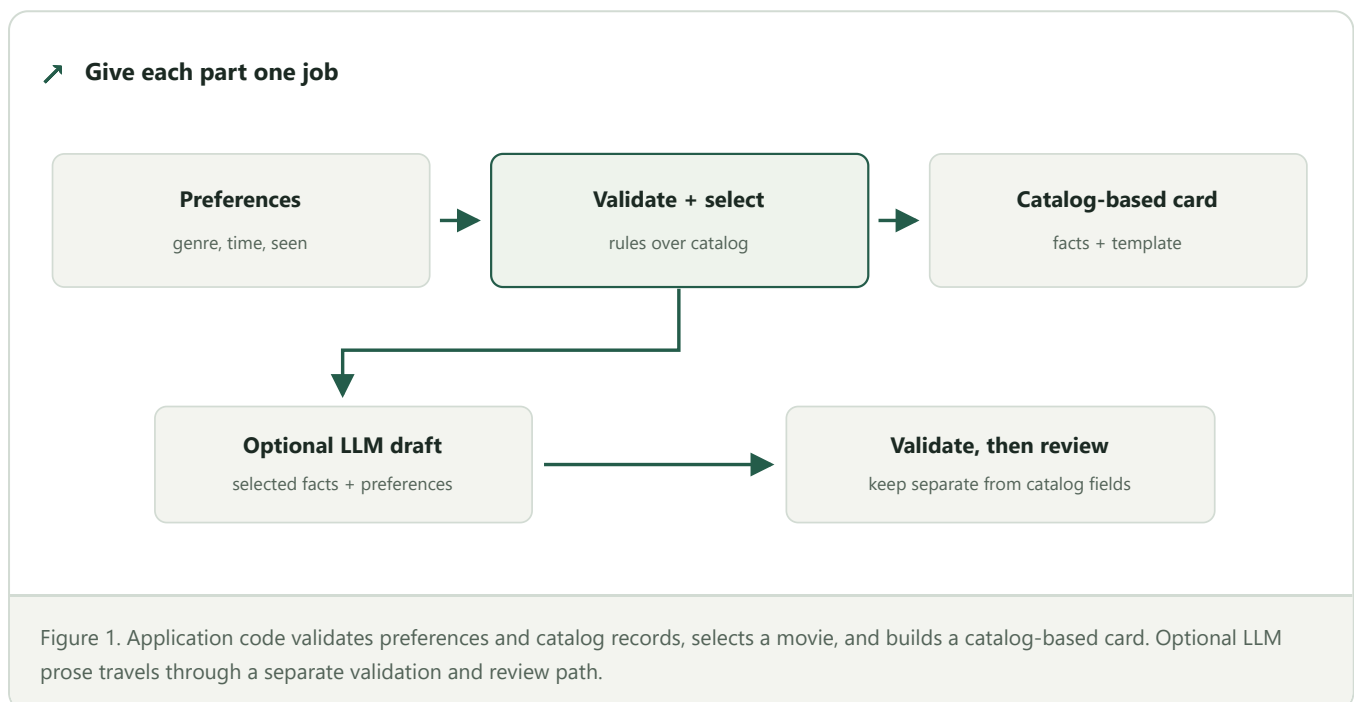
Reading path: follow the core sections, try the demo, then run the project. Expand the deeper notes when you want more detail.

1. Start with a small product promise

“Help me choose an unseen adventure movie that fits into 120 minutes. I would prefer something light.” This request gives us a task small enough to build and test.

The application returns one suggested movie, the catalog facts that explain the choice, and streaming availability if supplied. If nothing qualifies, it says so and lets the viewer change the request. It never promises that the viewer will enjoy the film.

An **acceptance criterion** is an observable condition for calling a feature successful. Here, the chosen movie must exist in the catalog, match the required genre, fit the time limit, and be absent from the viewer’s seen list. A missing streaming service must stay unknown. These conditions are more useful than “make the AI good.”



The first working version is a **baseline**: a simple reference implementation to compare improvements against. Our selection rules and template explanation need no trained model. Adding an LLM makes the explanation stage generative; it does not turn the ranking rule into learned personalization. This project connects Chapters 4-5’s software foundations with Chapters 9-10’s prompting and validation.

2. Keep catalog facts separate from preferences

The downloadable catalog has seven fictional movies. Each has a stable ID, title, genre tags, tone, runtime, and an optional streaming service. A **stable ID** identifies the same record even when its display title changes. For example:

```
{
  "id": "M001",
  "title": "Moonlight Map",
  "genres": ["adventure", "comedy"],
  "tone": "light",
  "runtime_minutes": 105,
  "streaming_service": null
}
```

Preferences are a different object: `genre`, `max_minutes`, `prefer_light`, and `seen_ids`. The viewer can change them without rewriting the movie's runtime. `null` means a value is unknown; it does not mean zero minutes or unavailable everywhere.

The catalog's `version` identifies this input snapshot. Recording it alongside the request helps explain why a result changed after an edit. All movie titles, tone labels, and service names in this project are invented for learning. They are not current cinema or streaming information.

▼ Go deeper: data validation is not factual verification

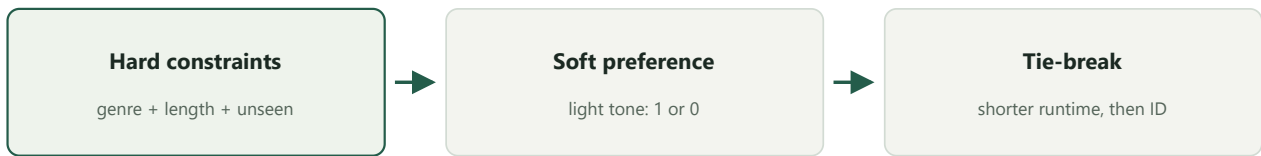
The project rejects duplicate movie IDs, unknown genre tags, unexpected fields, and invalid types. It accepts a null runtime as missing data. These checks make records usable by the program; they do not prove that a real-world film actually has that runtime. A real catalog also needs provenance, update dates, regional availability, and a process for correcting errors. We will develop data-quality concepts in Chapter 12.

3. Filter hard constraints before ranking

A **hard constraint** must hold for a result to qualify. Our hard constraints are the required genre, maximum runtime, and exclusion of seen movies. The "any" genre option removes only the genre restriction. "Maximum 120 minutes" includes a film that lasts exactly 120 minutes.

A **soft preference** helps choose among qualifying results. Here, a light tone is preferred, not required. A serious movie can still be returned if it is the best eligible option under our policy.

➤ **Filter first. Then compare eligible movies.**



A 140-minute movie cannot satisfy a 120-minute limit.

An unknown runtime cannot establish that the limit is satisfied either.

Figure 2. Hard constraints decide eligibility. The soft light-tone preference only orders eligible movies; it cannot rescue a movie that breaks a required constraint.

Moonlight Map passes the default request: it has the adventure tag, lasts 105 minutes, and is unseen. City Sprint lasts 140 minutes and has only the action tag, so it fails two constraints. Unlisted Voyage has an adventure tag but no runtime; the program cannot establish that it fits the time limit, so it is excluded too.

▼ **Go deeper: requirements should match the wording**

“I would prefer something light” allows a tradeoff. “Only show light movies” expresses a hard constraint and would need a separate filter. Likewise, “under 120 minutes” strictly means less than 120, while this project’s maximum-length control means at most 120. Translate wording into explicit rules before implementation. If an interface quietly changes the meaning, even perfectly executed code can solve the wrong problem.

4. Rank candidates with a transparent rule

A **candidate** is a movie still under consideration. **Ranking** puts candidates in a preferred order. Our small baseline assigns an eligible movie one point if light tone is preferred and the movie is light; otherwise it gets zero. Higher scores come first.

Eligible movie for the default request	Light-tone score	Runtime
Moonlight Map · M001	1	105 minutes
Forest Signal · M003	1	115 minutes
Quiet Orbit · M004	0	110 minutes

We break equal scores by shorter runtime, then movie ID. Moonlight Map wins over Forest Signal because 105 is less than 115. Quiet Orbit follows both because its score is lower, even though it is shorter than Forest Signal. This is a deliberate product policy, not a mathematical law about better movies.

The score is neither a probability nor a predicted rating. We have not trained anything or used other viewers' behavior. Large recommenders often separate candidate generation, scoring, and re-ranking; this project's filter-and-sort pipeline is a small, explicit design. See [Google's recommendation architecture overview](#).

▼ Go deeper: cold start, ties, and learning later

Cold start means making recommendations with little interaction history for a new viewer or item. Asking for preferences is one starting point. Later, ratings, watch behavior, or item features could support learned scoring, but you would need suitable data and evaluation. Even a learned score often needs explicit tie-breaks and product constraints. Stable sorting rules make this baseline reproducible; they do not make it personalized beyond the supplied preferences.

5. Try the complete recommendation flow

Use the demo below before reading the implementation. Start with the defaults. Then mark Moonlight Map as seen: Forest Signal becomes first. Turn off the light-tone preference as well: Quiet Orbit now wins because its 110-minute runtime is shorter than Forest Signal's 115 minutes. Finally, lower the adventure time limit to 90: nothing qualifies.

YOUR FIRST COMPLETE PROJECT

Local rules · fictional catalog

What should we watch?

Choose your preferences and inspect the recommendation. Everything here runs in your browser; the explanation uses a template.

Default run: adventure, no more than 120 minutes, prefer a light tone, no seen movies. Moonlight Map is 105 minutes, within your 120-minute limit. Genre: adventure, comedy. Tone: light. Selected by the stated ranking rule; enjoyment is not guaranteed. Streaming availability remains unknown.

Movie	Decision	Score
Moonlight Map	Eligible	1
City Sprint	Genre does not match; Over time limit	n/a
Forest Signal	Eligible	1
Quiet Orbit	Eligible	0
Cafe Chaos	Genre does not match	n/a
Winter Road	Genre does not match	n/a
Unlisted Voyage	Runtime unknown	n/a

Ranking: M001 → M003 → M004. At a 90-minute limit, no adventure qualifies. Excluding M001 at 120 minutes selects M003. Turning off the light preference in that last case selects M004.

The explanation is assembled from catalog fields and the request. It reports why the selected item qualifies, without inventing plot details or availability. The decision table exposes the rule's operation, not a model's private reasoning. Downloading the result saves an inspectable JSON snapshot; changing preferences alone does not save them.

6. Run and understand the Python project

Extract all files into one folder and open a terminal there. Python 3.10 or newer is sufficient; no third-party packages are needed for this workbook.

```
py movie_app.py
py evaluate.py
py -m unittest -v test_movie_app.py
```

Use your working `python` or `python3` command if `py` is unavailable. The first command prints a result with `status: ok`, movie ID M001, and an unknown streaming service. The other commands run the prepared evaluation and code tests. None calls a model service.

Open `preferences.json` and put `"M001"` in `seen_ids`; rerun the first command. The recommendation should change to M003. Set `prefer_light` to `false`; it should change to M004. These are the same decisions you explored in the browser.

The main `recommend` function validates inputs, creates an exclusion-reason list for each record, sorts eligible movies, and builds a result. Its sorting key is compact:

```
eligible.sort(key=lambda m: (
    -int(p["prefer_light"] and m["tone"] == "light"),
    m["runtime_minutes"],
    m["id"],
))
```

Python compares tuple elements from left to right. Negating the score puts one-point movies before zero-point movies in an ascending sort. Runtime and ID resolve ties. Missing runtimes have already been filtered out.

▼ Go deeper: where the browser and Python run

The handbook demo executes JavaScript locally; it does not send requests to the downloaded Python program. The workbook implements the same rules in Python, and automated checks compare their outputs. In a deployed application, a browser could send preferences to a server that owns validation, catalog access, and model credentials. You would then need an HTTP contract, loading and error states, and server-side validation. Chapter 5 introduced that boundary; this project keeps the two learning environments independent.

The folder's README maps each file. `movie_app.py` owns the project flow, while `llm_core.py`, `provider.py`, and `errors.py` reuse Chapter 10's service boundary. Use `movie_app.py` as the entry point.

7. Add an LLM explanation as a separate stage

An LLM can phrase the explanation more flexibly. It should receive the viewer preferences and the selected movie's facts, rather than being asked to invent an entire catalog. In this project, S1 holds preferences, S2 holds the selected movie, and S3 holds its supplied streaming information. M001 is a movie ID; S2 is a source ID within the current prompt.

First exercise the integration offline:

```
py movie_app.py --draft template
py movie_app.py --draft unsupported
```

The template mode constructs an authored response and sends it through Chapter 10's real envelope, JSON, field, and source-ID checks. It is not generated by an LLM. Unsupported mode deliberately inserts an ExampleFlix claim; it demonstrates that a false fact can pass structural checks.

The result keeps two separate objects. `recommendation` is the catalog-based card. `draft.candidate` contains the optional prose and the familiar four Chapter 10 fields. Its `review_required` status means a person must compare claims with the supplied records. The draft never overwrites the card's movie or streaming-service field. Valid structure still allows factual mistakes, as described in OpenAI's Structured Outputs guidance.

▼ Go deeper: optional live mode and the review boundary

With the local credentials and compatible model configuration described in Chapter 10, explicitly run `py movie_app.py --draft live`. This transmits the selected records and preferences and can incur API charges. Keep credentials outside the handbook browser and source files. The download README explains setup and limitations; no live API call was made for this edition.

The project makes at most one provider call for a requested draft and none when no movie qualifies. The browser demo remains offline. Check the draft's selected title, runtime, preference claims, source support, and availability before using its wording. Separating the draft limits its ability to corrupt catalog fields, but does not make its prose trustworthy. Prompt instructions alone are not evidence verification.

8. Make unhelpful outcomes understandable

An empty result is sometimes the correct answer. With adventure and a 90-minute maximum, "No matching movie" respects the viewer's request. Quietly increasing the limit would conceal a product decision the viewer did not make.

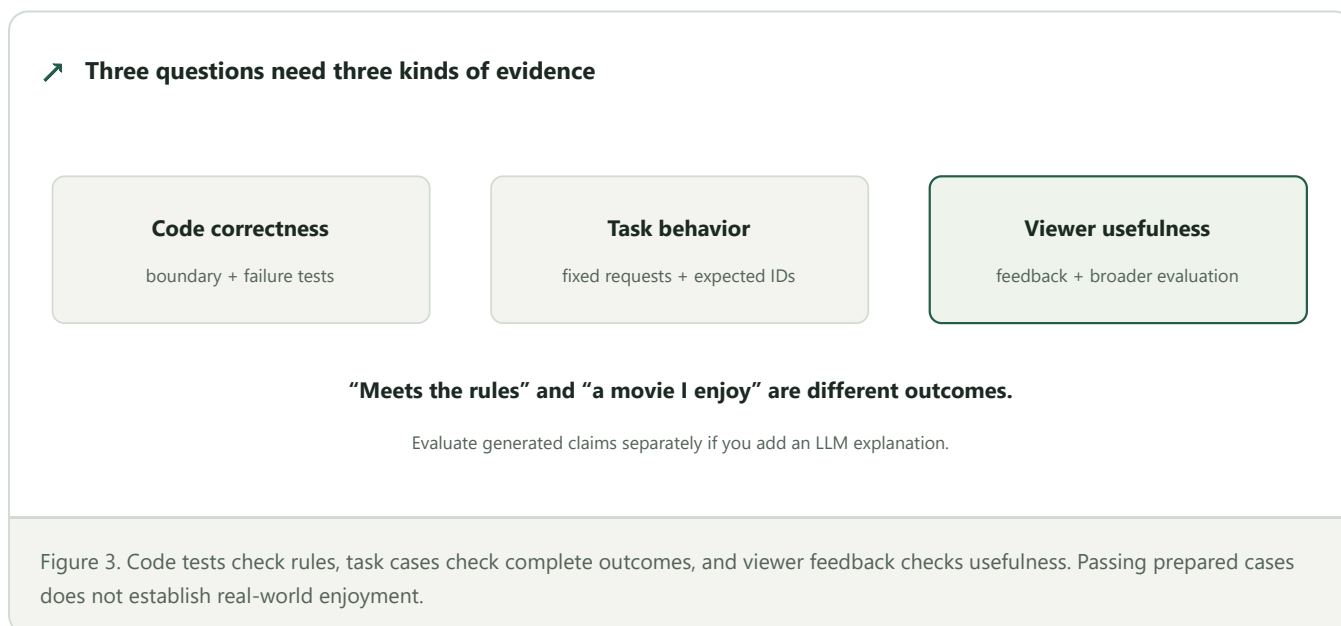
Situation	Project behavior
Invalid preference types or unknown IDs	Stop and describe the input problem
Valid request, no eligible movie	Return <code>no_match</code> ; suggest changing preferences
Streaming service missing	Keep <code>null</code> ; show “unknown in this catalog”
Optional provider fails	Keep the catalog-based card; record the draft failure
Draft passes structure checks	Keep it separate with <code>review_required</code>

Keeping a usable simpler result when an optional service fails is **graceful degradation**. The interface must distinguish that result from a successful model response. In the CLI, a normal result or no match exits 0; a requested draft failure exits 1; invalid input or setup exits 2. A successful exit code describes program behavior, not factual correctness of model prose.

9. Evaluate the product, not just the happy path

The workbook’s `eval_cases.json` contains 12 requests and expected movie IDs, including expected no-match results. These expectations were written for the stated policy; the recommender does not generate its own answer key. Running `evaluate.py` compares actual IDs with that file and reports each case.

The default catalog passes all 12 cases. That means **12 prepared policy checks passed**, not “100% recommendation accuracy.” Cases cover exact time boundaries, seen exclusions, soft-preference changes, genre tags, and empty results. They do not measure whether viewers enjoy the recommendations or whether a live LLM writes accurate explanations.



Code tests add different checks: every eligible movie respects the constraints across many time limits; duplicate IDs fail; missing runtimes are excluded; ties are stable; provider failures preserve the card; and no-match requests make zero provider calls. The tests use Python’s standard `unittest` framework.

▼ Go deeper: evaluating an improvement fairly

To compare an LLM explanation with the template, keep the same requests and catalog. Check factual support, correct handling of missing data, clarity, latency, and cost; collect viewer feedback for usefulness. Count unsupported claims separately from valid-JSON rates. A nicer sentence is not an improvement if it invents a service. These prepared cases are development checks. If you tune prompts or a learned ranker against them, preserve separate unseen cases for a less biased final assessment. Real availability also changes by date and region.

10. Practice: predict, change, and inspect

Exercise 1: separate a rule from a preference

The viewer requests adventure, at most 110 minutes, prefers light tone, and has seen M001. Which movie wins? Does its tone disqualify it?

▼ Reveal the result

M004, Quiet Orbit, wins. M001 is seen, M003 lasts 115 minutes, and M007 has no known runtime. M004 is an unseen 110-minute adventure, so it meets the hard constraints. Its serious tone gives it zero preference points but does not exclude it.

Exercise 2: expose an unsupported claim

Run `py movie_app.py --draft unsupported`. Compare `recommendation.streaming_service` with `draft.candidate.streaming_service`. Which value should a catalog-based “Where to watch” field display, and why?

▼ Reveal the distinction

The card must display unknown because its catalog field is null. The draft’s ExampleFlix claim is unsupported even though the JSON and source IDs passed checks. Keep the draft pending review; do not silently copy it into the card.

Exercise 3: prove the evaluation can fail

Back up `eval_cases.json`, then change the default case’s expected ID from M001 to M002. Run `py evaluate.py`, inspect the mismatch, and restore the original expectation. Why is changing the answer key until the score passes a poor evaluation practice?

▼ Reveal the lesson

The modified evaluation reports 11 of 12 cases passing and exits 1. The default case shows expected M002 and actual M001. Expectations should follow the requirements, not be adjusted merely to match implementation output. A deliberate policy change may require updated expectations, with the reason documented.

11. Check your understanding

Answer all five questions, then read the explanations.

01 A light adventure is 140 minutes long; the viewer has a 120-minute limit. What happens?

- A. Its light-tone score can rescue it.
- B. It is excluded before ranking.
- C. The app silently raises the time limit.

Answer: B. A required constraint decides eligibility. A soft preference only orders movies that already qualify.

02 A movie has no runtime in the catalog. Can it pass a maximum-length constraint?

- A. No. The app cannot establish that it fits.
- B. Yes. Missing runtime means zero minutes.
- C. Yes, if the title sounds short.

Answer: A. Unknown is a distinct data state. This project excludes the movie and records the reason instead of guessing its length.

03 Two eligible movies have the same light-tone score. Which rule applies next?

- A. Whichever was inserted first.
- B. The LLM invents a confidence score.
- C. Shorter runtime, then stable movie ID.

Answer: C. An explicit tie-break makes repeated runs reproducible. This is a chosen product rule, not a learned measure of enjoyment.

04 An optional LLM draft names ExampleFlix, but the catalog service is unknown. What should the UI do?

- A. Overwrite the catalog with ExampleFlix.
- B. Keep the catalog field unknown and flag the draft for evidence review.
- C. Treat valid JSON as verified availability.

Answer: B. The catalog-based card and model-written draft are separate. Structural validation does not prove the generated claim.

05 All 12 prepared evaluation cases pass. What does this establish?

- A. Every viewer will enjoy the recommendations.
- B. A personalized ML model has been trained.
- C. The implementation matches the expected outcomes for those cases.

Answer: C. These are small, authored acceptance cases for a deterministic policy. They do not measure live model quality or real viewer satisfaction.

12. Keep the boundaries visible

Validate the inputs. Filter required constraints. Rank eligible choices. Explain from supplied facts. Review model prose separately. Test the cases where the answer is missing or the service fails.

You have completed a small product with an inspectable policy, a working interface, a Python implementation, and repeatable checks. You can now explain which behavior comes from data, which comes from application rules, and where an optional model enters the system.

▼ Quick-reference glossary

Term	Meaning in this project
Acceptance criterion	An observable condition the product should satisfy
Baseline	A simple reference implementation for comparison
Hard constraint	A requirement that determines eligibility
Soft preference	A criterion for ordering eligible options
Candidate / ranking	An item under consideration / its ordering against others
Tie-break	A further rule for equally scored items
Cold start	Making recommendations with little interaction history
Catalog version	The input snapshot used for this run
Graceful degradation	Preserving a useful simpler result when an optional part fails
Acceptance case	A prepared request with an expected product outcome

Sources linked above support the architecture, structured-output limits, and testing tools. The catalog, ranking policy, interface, diagrams, and exercises are original learning examples.

Next: Chapter 12 - Preparing data for machine learning. We move from consuming a small clean catalog to inspecting, cleaning, and splitting data without leaking information into evaluation.